Reinforcement Learning Project

Hédi Hadiji

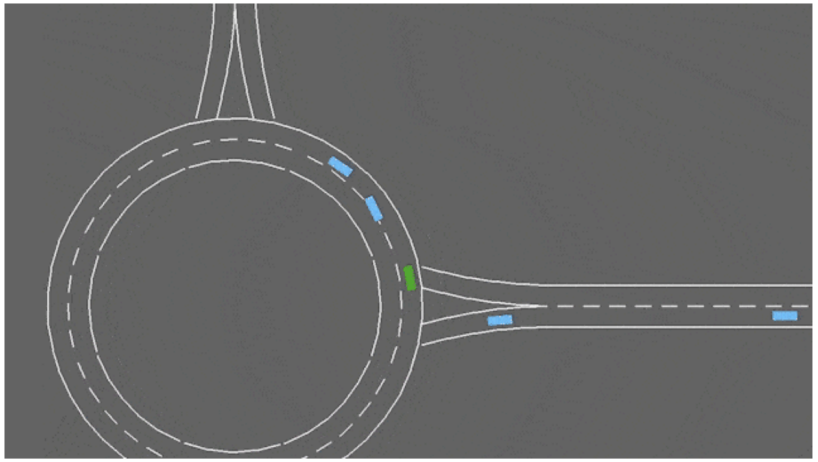
CentraleSupelec
hedi.hadiji@l2s.centralesupelec.fr

Mention IA

Goal

http://highway-env.farama.org/

Training curves, for 5 random seeds.



Train and analyze RL agents in the Highway environment collection.

General format

- groups of exactly 4 students
- one shared repository per group
- one individual report per student (2–4 pages)
- one 10-minute oral presentation per group

The project grade is primarily a group grade, with only minor individual adjustments based on the reports and the oral.
The project has two parts:

- one **shared core task**
- one **extension task**

Shared core task

All groups solve the same benchmark:

- environment: `highway-v0`
- configuration: instructor-provided `shared_core_config.py`
- observations: `Kinematics`
- actions: `DiscreteMetaAction`
- one DQN implemented by yourselves
- one model trained with Stable-Baselines

Core task requirements

For the core task, you must:

- implement a DQN agent yourselves
- train it on the shared benchmark
- train one Stable-Baselines model on the same benchmark
- compare the two approaches
- save checkpoints and metrics
- evaluate thoroughly (50 runs mean rewards + std)
- analyze the learned behavior and at least one failure mode

What to include for the core task

Minimum evidence:

- training curves
- evaluation table over at least 3 seeds
- comparison between your DQN and the Stable-Baselines model using the same config, same evaluation seeds, and same metrics
- one recorded rollout or qualitative behavior analysis
- one failure case with an explanation
- a short discussion of design choices

Extension task

Each group completes one **substantial** extension.
A small tuning exercise by itself is not enough.
Possible directions:

- continuous-action environment
- effect of observation representation
- effect of reward shaping
- effect of network architecture
- Prioritized Experience Replay, Double-DQN, other algorithmic ameliorations
- generalization to modified traffic or reward settings
- safety-oriented training or evaluation (e.g. can we reach strong performances with less crashes?)

What counts as a strong extension?

A good extension should answer a real question.
It should contain:

- a clear objective or hypothesis
- a non-trivial experimental design
- a comparison, ablation, or controlled variation
- an honest interpretation of the outcome

Deliverables

Group deliverables:

- one repository with code and a short README
- one common summary of environment, config, and main results
- one 10-minute oral presentation

Individual deliverables:

- one report per student
- one clearly identified personal contribution

Individual report

Each report should contain:

- what you implemented or analyzed
- what experiments you ran
- what worked and what failed
- evidence of your contribution
- a short discussion of limitations

Regarding the use of AI

Reasonable use of language models is authorized

Examples of unreasonable uses:

- emojis in your code
- 'tutoiement'/ commments to the coder (e.g. '# Ton dataloader ')
- excessively defensive coding patterns and handling of edge cases (try from future import)
- excessive commenting: 'e.g. 'print(x) # this prints x '

Grading

Out of 20:

- 6 points: core task quality
- 6 points: extension quality
- 3 points: individual report
- 5 points: oral presentation

Practical advice

- start from the common benchmark first
- keep the training pipeline reproducible
- save configs, seeds, and plots
- do not hide failed attempts if they taught you something
- divide work clearly, but keep a shared understanding of the full project